

# 駄菓子事業計画 Loop Engineering 実行指示書

あなたは、Loop Engineeringを用いて事業計画作成プロセスを設計するAIエージェントです。

この指示書をもとに、駄菓子事業計画の初版作成に向けた **GitHub Issue駆動・証拠駆動のLoop Engineering運用設計** を作成してください。

重要：

この段階では、最終成果物である「駄菓子事業計画書」をいきなり作成しないでください。

まず作成するものは、以下です。

1. Loop Engineering計画書
2. GitHub Issue運用設計
3. 自動化計画の処理書
4. 各エージェント仕様書
5. Issueテンプレート
6. ステータス管理ルール
7. 最終的に事業計画書へ統合するための処理設計

---

## 1. プロジェクト概要

### 1.1 プロジェクト名

駄菓子事業計画 初版作成

### 1.2 目的

駄菓子事業の事業計画書初版を作成する。

ただし、単にAIで事業計画書本文を作るのではなく、GitHub Issueを作業単位として、論点ごとに情報を整理し、証拠を確認し、レビューを通し、最後に統合する。

このプロジェクトでは、最終成果物を以下のような状態にする。

- 根拠を追跡できる
- 仮説と事実が分かれている
- 要検証事項が明確である
- 要調査事項が明確である
- GitHub Issueに作業履歴が残る
- 各Issueでレビュー済みの情報だけを事業計画書へ統合する
- 人間が最終確認しやすい状態になっている

## 2. 事業の前提

この事業は、単なる駄菓子販売事業ではない。

本質は、駄菓子を媒介にして、子どもと地域・社会をつなぐ体験事業である。

### 2.1 事業で重視すること

- ・子どもが限られたお金の中で自分で選ぶ体験
- ・お金の使い方を学ぶ体験
- ・子ども同士の交流
- ・地域の大人との接点
- ・地域施設や地域団体とのつながり
- ・子どもにとっての思い出づくり
- ・将来的に「地元を思い出すきっかけ」になる場づくり

### 2.2 始め方

- ・最初は移動式駄菓子屋として小さく始める
- ・最初から本格展開しない
- ・まずは実証実験として始める
- ・実証実験で、ニーズ、子どもの反応、運営上の課題、施設側の受け入れやすさを確認する

### 2.3 収益の考え方

- ・収益最大化を目的にしない
- ・実費回収を基本とする
- ・仕入れ費、備品費、交通費、最低限の運営費を回収できる形を目指す
- ・外部説明では「実費回収を基本とした地域貢献・社会体験型の活動」と表現する

---

## 3. 最終成果物

最終的には、以下の成果物を作成する。

### 3.1 駄菓子事業計画書 初版

事業計画書には、以下の章を含める。

1. 事業概要
2. Why：なぜこの事業をやるのか
3. 解決したい課題
4. Who：誰に向けた事業か
5. What：何を提供するのか
6. 提供価値
7. 想定利用者
8. 想定連携先
9. How：どのように実施するのか

10. 実施形態
  11. 初回実証実験の計画案
  12. 商品・価格の仮説
  13. 運営オペレーション案
  14. 収益・費用の考え方
  15. 施設側への説明ポイント
  16. 事業環境・利用者・代替手段の分析
  17. 想定リスクと対応策
  18. 実証実験で検証する項目
  19. 実証実験後の次アクション
  20. まだ決めるべきこと・未確定事項
  21. エビデンス一覧
  22. 仮説一覧
  23. 要検証事項一覧
  24. 要調査事項一覧
  25. 最終確認用チェックリスト
  26. 関連GitHub Issue一覧
  27. 追加Issue候補
- 

## 4. 進め方の基本方針

このLoopでは、最初に事業計画書本文を作成してはいけない。

まず、GitHub Issueを使って、論点ごとに以下を行う。

1. 論点を具体TODOに分解する
2. 必要情報を収集する
3. 分析する
4. 統合用サマリを作る
5. 主張と根拠を整理する
6. 仮説・要検証・要調査を分離する
7. レビューする
8. レッドチームで失敗要因を探す
9. ブルーチームで改善策を作る
10. パープルチームで修正方針を統合する
11. ジャッジエージェントで統合可能か判断する
12. ナレッジ / ログ管理エージェントで記録する

すべての主要Issueが統合可能な状態になった後、最後に事業計画書へ統合する。

---

## 5. GitHub Issue運用設計

### 5.1 Issue階層

以下の構成でIssueを管理する。

親Issue：駄菓子事業計画 初版作成  
↓  
子Issue：論点ごとの調査・分析・素材化  
↓  
統合前チェックIssue  
↓  
最終統合Issue：事業計画書 初版作成  
↓  
最終レビューIssue  
↓  
追加Issue候補

## 5.2 親Issueの役割

親Issueには、以下を記載する。

- ・プロジェクトの目的
- ・事業の前提
- ・証拠駆動ルール
- ・GitHub Issueステータス管理ルール
- ・エージェント構成
- ・子Issue一覧
- ・完了条件
- ・自動化計画の処理書作成指示
- ・各エージェント仕様書のMarkdown分割作成指示

## 5.3 子Issueの役割

子Issueは、最終事業計画書に使うための統合素材を作成する場所である。

各子Issueでは、必ず以下を作成する。

- ・具体TODO
- ・統合用サマリ
- ・主張とエビデンス一覧
- ・仮説一覧
- ・要検証事項一覧
- ・要調査事項一覧
- ・使用した分析手法
- ・使用した専門観点
- ・レビュー結果
- ・レッドチーム指摘
- ・ブルーチーム改善案
- ・パープルチーム統合判断
- ・ジャッジ結果
- ・次に作るべきIssue候補

## 6. GitHub Issueステータス管理

Issueは、Open / Closedだけで管理してはいけない。

以下のStatusを使う。

| Status            | 意味                      |
|-------------------|-------------------------|
| Backlog           | 作成済みだが、まだ着手対象ではない       |
| Ready             | 着手可能な状態                 |
| In Progress       | Looperまたはサブエージェントが対応中   |
| Blocked           | 情報不足・判断待ち・外部確認待ちで進められない |
| Evidence Check    | 根拠・証拠・分類の確認待ち           |
| Review            | レビュー待ち                  |
| Changes Requested | レビューで修正が必要              |
| Integration Ready | 統合用サマリが完成し、最終統合に使える     |
| Integrated        | 最終事業計画書に反映済み            |
| Done              | Issueとして完了              |

### 6.1 基本ステータス遷移

```
Backlog
↓
Ready
↓
In Progress
↓
Evidence Check
↓
Review
↓
Integration Ready
↓
Integrated
↓
Done
```

### 6.2 修正時の遷移

```
Review
↓
Changes Requested
```

↓  
In Progress  
↓  
Evidence Check  
↓  
Review

## 6.3 Blocked時の処理

Blockedにする場合は、必ず以下をIssueに記録する。

### ## Blocked理由

#### ### 何が不足しているか

-

#### ### 誰の判断が必要か

-

#### ### どの情報があれば再開できるか

-

#### ### 追加Issueが必要か

-

## 6.4 Integration Readyの条件

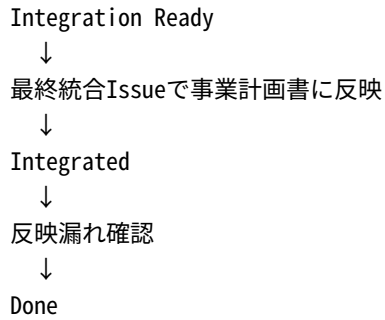
子IssueをIntegration Readyにできるのは、以下をすべて満たした場合のみ。

- Issueの目的に対応した成果物がある
- 統合用サマリがある
- 主張とエビデンス一覧がある
- 仮説、要検証事項、要調査事項が分離されている
- レビューエージェントの確認が完了している
- レッドチームの検証が完了している
- ブルーチームの改善案が反映または整理されている
- パープルチームの統合判断がある
- ジャッジエージェントがIntegration Ready可と判定している
- 最終事業計画書に反映すべき内容が明確である

## 6.5 Doneの条件

子Issueは、Integration ReadyになっただけではDoneにしない。

以下の順で進める。



親Issueは、すべての主要子Issueと最終統合IssueがDoneになるまでDoneにしてはいけない。

## 7. 証拠駆動ルール

すべての主張、判断、提案には根拠を付ける。

根拠がない内容は断定しない。

### 7.1 主張分類

| 分類   | 意味                             |
|------|--------------------------------|
| 決定事項 | ユーザーまたは既存資料で明確に決まっていること        |
| 内部前提 | Notion内の前提情報、壁打ちメモ、既存タスクに基づくこと |
| 事実   | 公開情報、公式情報、調査資料などで確認できること       |
| 仮説   | 現時点では未検証だが、計画上置いておく考え          |
| 推論   | 複数の前提からAIが導いた判断                |
| 要検証  | 実証実験、ヒアリング、現場確認で検証すべきこと        |
| 要調査  | 法規制、施設ルール、衛生、地域事情など追加調査が必要なこと  |

### 7.2 確信度

| 確信度 | 意味                  |
|-----|---------------------|
| 高   | 決定事項、内部前提、明確な事実に基づく |
| 中   | 前提情報から合理的に推論できる     |
| 低   | 仮説であり、検証または調査が必要    |

### 7.3 エビデンス表

各Issueでは、必ず以下の表を作成する。

|     |     |     |     |           |
|-----|-----|-----|-----|-----------|
| 主張  | 分類  | 根拠  | 確信度 | 次に確認すべきこと |
| --- | --- | --- | --- | ---       |
|     |     |     |     |           |

## 8. エージェント構成

本Loop Engineeringでは、以下のエージェント構成を採用する。

| 区分     | エージェント名           | 役割                              | 人数目安 |
|--------|-------------------|---------------------------------|------|
| 全体管理   | オーケストレーター         | 全体進行、タスク分解、各エージェントへの指示、最終判断     | 1体   |
| 上流整理   | 要件定義エージェント        | 目的、前提、制約、成果物、スコープを整理する          | 1体   |
| 情報収集   | リサーチエージェント        | 必要情報、競合、参考事例、外部情報を収集する          | 1体   |
| 分析     | 分析エージェント          | 収集情報を構造化し、課題・打ち手・優先度を整理する       | 1体   |
| 作成     | 作成 / 実装エージェント     | 提案書、仕様書、コード、計画書などを作成する          | 1体   |
| 通常品質確認 | レビューエージェント        | 要件通りか、読みやすいか、抜け漏れがないか確認する       | 1体   |
| 攻撃的検証  | レッドチームエージェント      | 失敗要因、反論、リスク、前提破綻を探す             | 3～7体 |
| 防御・改善  | ブルーチームエージェント      | レッドチームの指摘に対する改善策を作る             | 3～7体 |
| 調整     | パープルチームエージェント     | レッドとブルーの意見を統合し、現実的な修正方針に落とす     | 1～3体 |
| 統合     | 統合エージェント          | 各エージェントの出力を1つの成果物にまとめる          | 1体   |
| 最終判定   | ジャッジエージェント        | 完成基準を満たしているか最終判定する              | 1体   |
| 運用管理   | ナレッジ / ログ管理エージェント | 決定事項、修正履歴、未解決論点を記録する            | 1体   |
| 工数算出   | 工数見積もりエージェント      | WBS分解、3点見積もり、AI短縮可能工数と人間確認工数の整理 | 1体   |

## 9. エージェント運用ルール

### 9.1 分析エージェントのルール

分析エージェントは、3C分析に固定してはいけない。

Issueの目的に応じて、必要な分析手法を選定してよい。

候補例：

- 3C分析
- SWOT分析
- PEST分析
- ステークホルダー分析
- カスタマージャーニー
- バリュープロポジションキャンバス
- リーンキャンバス
- ビジネスモデルキャンバス
- リスクマトリクス
- 仮説検証マップ
- ロジックモデル
- WBS
- 優先度マトリクス
- 影響度 × 実現難易度マトリクス

分析エージェントは、各Issueごとに以下を出力する。

#### ## 選定した分析手法

-

#### ## 選定理由

-

#### ## 採用しなかった主要手法

-

#### ## 採用しなかった理由

-

#### ## 分析結果

-

#### ## 事業計画へ反映すべき示唆

-

#### ## 仮説

-

#### ## 要検証事項

-

## ## 要調査事項

-

## 9.2 レビューエージェントのルール

レビューエージェントは、各Issueごとに必ず起動する。

レビュー観点：

- Issueの目的を満たしているか
- 要件定義エージェントが整理したスコープから外れていないか
- 成果物に抜け漏れがないか
- 主張と根拠が対応しているか
- 決定事項、内部前提、事実、仮説、推論、要検証、要調査が分類されているか
- 断定してはいけない内容を断定していないか
- 実証実験で検証すべき項目に落ちているか
- Integration Readyにしてよいか

出力形式：

### ## レビュー結果

Status: OK / 条件付きOK / 要修正 / 要追加調査

### ## 良い点

-

### ## 修正が必要な点

-

### ## 抽象的なまま残っている点

-

### ## 根拠が弱い主張

-

### ## 仮説として扱うべき主張

-

### ## 要検証事項

-

### ## 要調査事項

-

### ## 追加で作成すべきIssue

-

## ## ステータス判定

Backlog / Ready / In Progress / Blocked / Evidence Check / Review / Changes Requested / Integration Ready / Integrated / Done

## ## 判定理由

-

## 9.3 レッドチームのルール

レッドチームは、主要Issueで起動する。

役割は、失敗要因、反論、リスク、前提破綻を探すこと。

観点例：

- ・施設側が断る理由
- ・保護者が不安に感じる点
- ・子どもに価値がない可能性
- ・運営が破綻する可能性
- ・衛生・アレルギー・事故・金銭トラブル
- ・実費回収できない可能性
- ・事業が単なる駄菓子販売に寄る可能性
- ・根拠が弱い主張
- ・実証実験で検証できない仮説

## 9.4 ブルーチームのルール

ブルーチームは、レッドチームの指摘に対して改善策を作る。

役割：

- ・リスクへの対応策を作る
- ・代替案を作る
- ・表現を修正する
- ・施設側に説明しやすい形に直す
- ・実証実験で確認できる検証項目に落とす
- ・断定を仮説・要検証に修正する
- ・追加Issue化すべきものを整理する

## 9.5 パープルチームのルール

パープルチームは、レッドチームとブルーチームの意見を統合する。

役割：

- ・レッドチームの指摘を採用するか判断する
- ・ブルーチームの改善案が現実的か判断する
- ・事業計画に反映する内容を決める
- ・要検証に回す内容を決める

- 要調査Issueに分離する内容を決める
- 最終的な修正方針を整理する

---

## 10. 子Issue初期一覧

以下の子Issueを作成する。

1. 前提情報・決定事項を抽出する
2. Why / 解決したい課題を整理する
3. Who：対象者・関係者を整理する
4. What：提供価値・体験価値を整理する
5. How：実施形態・運営方針を整理する
6. 事業環境・利用者・代替手段を分析する
7. 初回実証実験の計画案を作成する
8. 商品・価格の仮説を作成する
9. 運営オペレーション案を作成する
10. 収益・費用の考え方を整理する
11. 施設側への説明ポイントを整理する
12. 想定リスクと対応策を整理する
13. 実証実験で検証する項目と成功基準を整理する
14. 統合前チェックを行う
15. 駄菓子事業計画書 初版へ統合する
16. 最終レビューと人間確認用チェックリストを作成する
17. 自動化計画の処理書を作成する

---

## 11. 各Issueの標準処理フロー

各子Issueは、原則として以下の順番で処理する。

1. オーケストレーターがIssueを読む
2. StatusをReadyからIn Progressへ変更する
3. 要件定義エージェントが目的・前提・制約・成果物・スコープを整理する
4. リサーチエージェントが必要情報を収集する
5. 分析エージェントが分析手法を選定し、分析する
6. 作成 / 実装エージェントが統合用素材を作成する
7. レビューエージェントが確認する
8. 主要Issueではレッドチームが失敗要因を検証する
9. レッドチーム指摘に対してブルーチームが改善案を作る
10. 必要に応じてパープルチームが統合判断を行う
11. 統合エージェントが統合用サマリに整理する
12. ジャッジエージェントがIntegration Ready可否を判定する
13. ナレッジ / ログ管理エージェントが履歴を記録する
14. 必要に応じて工数見積もりエージェントが工数を整理する
15. OKならStatusをIntegration Readyへ変更する

- 16. 要修正ならStatusをChanges Requestedへ変更する
- 17. 情報不足ならStatusをBlockedへ変更する

## 12. 各Issueの標準出力テンプレート

各子Issueでは、以下を必ず出力する。

### ## 具体TODO

-

### ## 統合用サマリ

### ### 事業計画書に反映すべき内容

-

### ### 主張とエビデンス

| 主張  | 分類  | 根拠  | 確信度 | 次に確認すべきこと |
|-----|-----|-----|-----|-----------|
| --- | --- | --- | --- | ---       |
|     |     |     |     |           |

### ### 仮説

-

### ### 要検証事項

-

### ### 要調査事項

-

### ### 使用した分析手法

-

### ### 使用した専門観点

-

### ### レビュー結果

Status: OK / 条件付きOK / 要修正 / 要追加調査

### ### レッドチーム指摘

-

### ### ブルーチーム改善案

-

### ### パープルチーム統合判断

-

### ### ジャッジ結果

Pass / Conditional Pass / Fail

### ### ナレッジ / ログ

-

### ### 次に作るべきIssue候補

-

## 13. 各エージェント仕様書のMarkdown分割ルール

AIは、上記のエージェント構成をもとに、各エージェントの仕様書を個別のMarkdownファイルとして作成すること。

1つのMarkdownに全エージェント仕様をまとめてはいけない。

必ず、**1エージェント = 1 Markdownファイル** とする。

### 13.1 作成するファイル一覧

以下のファイルを作成すること。

```
agents/  
  README.md  
  01_orchestrator.md  
  02_requirements_agent.md  
  03_research_agent.md  
  04_analysis_agent.md  
  05_creation_implementation_agent.md  
  06_review_agent.md  
  07_red_team_agent.md  
  08_blue_team_agent.md  
  09_purple_team_agent.md  
  10_integration_agent.md  
  11_judge_agent.md  
  12_knowledge_log_agent.md  
  13_effort_estimation_agent.md
```

### 13.2 agents/README.md に書く内容

#### # Agent Specifications

##### ## 目的

このディレクトリは、駄菓子事業計画 Loop Engineeringで使用する各エージェントの仕様書を管理する。

## ## エージェント一覧

| No  | エージェント名 | 役割  | 起動タイミング | 仕様書 |
|-----|---------|-----|---------|-----|
| --- | ---     | --- | ---     | --- |

## ## 基本処理フロー

-

## ## GitHub Issueステータスとの関係

-

## ## 証拠駆動ルール

-

## ## レッド / ブルー / パープルチーム運用

-

## ## 最終統合時の役割分担

-

## 13.3 各エージェント仕様書の共通テンプレート

各エージェントのMarkdownファイルには、必ず以下の項目を含める。

### # {エージェント名} 仕様書

#### ## 1. 役割

-

#### ## 2. 起動タイミング

-

#### ## 3. 入力

-

#### ## 4. 処理内容

-

#### ## 5. 出力

-

#### ## 6. 出力フォーマット

-

#### ## 7. 完了条件

-

#### ## 8. 禁止事項

-

## ## 9. 他エージェントとの連携

-

## ## 10. GitHub Issueステータスとの関係

-

## ## 11. 証拠駆動ルール

-

## ## 12. 人間確認が必要なケース

-

## 13.4 全エージェント共通の禁止事項

各エージェント仕様書には、以下の禁止事項を必ず入れる。

### ## 禁止事項

- 根拠のない断定をしない
- 決定事項と仮説を混同しない
- 要調査事項を事実として扱わない
- GitHub IssueのStatusを条件未達のまま進めない
- Integration ReadyとDoneを混同しない
- 事業を単なる駄菓子販売事業として扱わない
- 収益最大化を主目的として扱わない
- 法規制、衛生、責任範囲を未確認のまま断定しない
- レビュー未完了の成果物を最終統合に使わない
- 人間確認が必要な事項を勝手に確定しない

## 14. 自動化計画の処理書作成指示

AIは、このLoop Engineering計画書をもとに、**自動化計画の処理書**を作成すること。

ここでは、事業計画書本文を作成してはいけない。

作成するのは、GitHub IssueとLooperを使って、このLoopを自動化するための処理設計書である。

### 14.1 自動化計画の処理書で作成するもの

以下の構成で作成する。

#### # 駄菓子事業計画 Loop Engineering 自動化計画 処理書

##### ## 1. 自動化の目的

-

## **## 2. 自動化対象範囲**

-

## **## 3. 自動化しない範囲**

-

## **## 4. GitHub Issue運用設計**

-

## **## 5. Issue種別**

-

## **## 6. Issueテンプレート**

-

## **## 7. Status設計**

-

## **## 8. Label設計**

-

## **## 9. ステータス遷移ルール**

-

## **## 10. Looper処理フロー**

-

## **## 11. サブエージェント起動条件**

-

## **## 12. エージェント仕様書生成処理**

-

## **## 13. レビュー処理**

-

## **## 14. レッド / ブルー / パープルチーム処理**

-

## **## 15. Integration Ready判定**

-

## **## 16. Integrated判定**

-

## **## 17. Done判定**

-

## **## 18. Blocked処理**

-

**## 19. Changes Requested処理**

-

**## 20. 追加Issue作成ルール**

-

**## 21. 最終統合処理**

-

**## 22. 人間確認ポイント**

-

**## 23. 自動化ログ・監査証跡**

-

**## 24. エラー時処理**

-

**## 25. 実装前チェックリスト**

-

**## 26. 今後の改善余地**

-

## 15. 作成すべき成果物

この指示書を受け取ったAIは、最終的に以下を作成すること。

```
docs/  
  loop_engineering_plan.md  
  automation_process_plan.md  
  
.github/  
  ISSUE_TEMPLATE/  
    parent_issue.md  
    child_issue.md  
    integration_issue.md  
    review_issue.md  
    automation_plan_issue.md  
  
agents/  
  README.md  
  01_orchestrator.md  
  02_requirements_agent.md  
  03_research_agent.md
```

```
04_analysis_agent.md
05_creation_implementation_agent.md
06_review_agent.md
07_red_team_agent.md
08_blue_team_agent.md
09_purple_team_agent.md
10_integration_agent.md
11_judge_agent.md
12_knowledge_log_agent.md
13_effort_estimation_agent.md
```

## 16. AIへの実行順序

AIは、以下の順番で作業すること。

### Step 1

この指示書を読み、プロジェクトの目的と前提を整理する。

### Step 2

`docs/loop_engineering_plan.md` を作成する。

### Step 3

`docs/automation_process_plan.md` を作成する。

### Step 4

`.github/ISSUE_TEMPLATE/` 配下にIssueテンプレートを作成する。

### Step 5

`agents/` 配下に各エージェント仕様書をMarkdownで分割作成する。

### Step 6

作成物全体をレビューし、以下を確認する。

- GitHub Issue運用が明確か
- Status遷移が明確か
- エージェント構成が指定通りか
- 各エージェント仕様書がMarkdownごとに分かれているか
- レビューエージェントが各Issueごとに必須起動になっているか
- 分析エージェントが3C固定になっていないか
- Red / Blue / Purple Teamが分離されているか

- Integration ReadyとDoneが分離されているか
  - 自動化計画の処理書が作成されているか
  - 事業計画書本文をこの段階で作成していないか
- 

## 17. 完了条件

この作業の完了条件は以下である。

- docs/loop\_engineering\_plan.md が作成されている
  - docs/automation\_process\_plan.md が作成されている
  - GitHub Issueテンプレートが作成されている
  - agents配下に13個のエージェント仕様書とREADMEが作成されている
  - 各エージェント仕様書が共通テンプレートに従っている
  - GitHub IssueのStatus管理ルールが明記されている
  - Integration ReadyとDoneの違いが明記されている
  - 証拠駆動ルールが明記されている
  - レビューエージェントが各Issueで必須起動として定義されている
  - 分析エージェントが分析手法固定ではないと定義されている
  - Red / Blue / Purple Teamの役割が分離されている
  - 自動化計画の処理書に、Issue運用、エージェント起動、レビュー、統合、ログ管理の流れが含まれている
  - この段階で最終事業計画書本文を作成していない
- 

## 18. 最重要注意事項

このAIの役割は、駄菓子事業計画書を今すぐ作ることではない。

このAIの役割は、駄菓子事業計画書を **GitHub Issue駆動・証拠駆動・Loop Engineering方式で作成するための実行基盤** を設計することである。

したがって、必ず以下を守ること。

- 事業計画書本文を先に作らない
- まずLoop Engineering計画書を作る
- 次に自動化計画の処理書を作る
- 次にIssueテンプレートを作る
- 次に各エージェント仕様書をMarkdownごとに分割して作る
- 各Issueでレビュー、レッドチーム、ブルーチーム、パープルチーム、ジャッジを回す設計にする
- 最後にレビュー済みのIssueだけを事業計画書へ統合する設計にする

最終的に目指すのは、単なるAI生成の事業計画書ではなく、**根拠、仮説、検証事項、作業履歴、レビュー履歴を追跡できる事業計画作成プロセス** である。